

**GOVRAG-TN: A Domain-Adaptive Cross-Lingual Retrieval
Augmented Generation Architecture for Trustworthy
Interpretation of Tamil Nadu Government Orders**

A PROJECT REPORT

Submitted by

DARISH M [Reg No: PA2312049010073]

Under the Guidance of

Dr. SELVAKUMARASAMY

Associate Professor, Department of Computational Intelligence

In partial fulfilment of the requirements for the degree of

MASTER OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE



**DEPARTMENT OF COMPUTATIONAL INTELLIGENCE
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203
MAY 2026**

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY****KATTANKULATHUR – 603 203****BONAFIDE CERTIFICATE**

Certified that this M.Tech project report titled “**GOVRAG-TN: A DOMAIN-ADAPTIVE CROSS-LINGUAL RETRIEVAL AUGMENTED GENERATION ARCHITECTURE FOR TRUSTWORTHY INTERPRETATION OF TAMIL NADU GOVERNMENT ORDERS**” is the bonafide work of **Mr. Darish M [Reg. No.: PA2312049010073]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion for this or any other candidate.

Dr. SELVAKUMARASAMY**SUPERVISOR**

Associate Professor

Department of Computational Intelligence

Dr. R. ANNIE UTHRA**HEAD OF THE DEPARTMENT**

Professor

Department of Computational
Intelligence**INTERNAL EXAMINER****EXTERNAL EXAMINER**



DEPARTMENT OF COMPUTATIONAL INTELLIGENCE

SRM Institute of Science and Technology Own Work

Declaration Form

Degree/ Course : M.Tech in Artificial Intelligence

Student Names : Darish M

Registration Number : PA2312049010073

Title of Work : GOVRAG-TN: A Domain-Adaptive Cross-Lingual Retrieval Augmented Generation Architecture for Trustworthy Interpretation of Tamil Nadu Government Orders

I hereby declare that this assessment has been prepared in accordance with the University's rules and regulations related to academic integrity, plagiarism, and misconduct, as specified in the University regulations and official guidelines.

I confirm that the work presented in this assessment is original, except where proper acknowledgement has been provided. I further certify that:

- All sources used have been properly cited and referenced
Quoted text from books, journals, websites, or other materials has been clearly indicated
- Sources for figures, tables, images, and datasets not belonging to me have been acknowledged
- No part of this work has been copied from reports or assignments of other students
- Any assistance received from individuals or external resources has been appropriately acknowledged
- All plagiarism and academic conduct requirements specified by the University have been followed

I understand that any false declaration or violation of academic integrity policies may result in action as per university regulations.

DECLARATION:

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my own work, except where indicated by referring, and that I have followed the good academic practices noted above.

Darish M PA2312049010073

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for providing the facilities and academic environment necessary for carrying out this project work. I also thank **Dr. M. Leenus J Martin**, Dean–CET, SRM Institute of Science and Technology, for his continuous support and encouragement.

I extend my heartfelt thanks to **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project. I am grateful to **Dr. R. Annie Uthra**, Professor and Head, Department of Computational Intelligence, SRM Institute of Science and Technology, for her guidance and encouragement during this work.

I sincerely thank **Dr. S. Ganesh Kumar**, Professor, Department of Data Science and Business Systems, and Program Coordinator, SRM Institute of Science and Technology, for his valuable suggestions and motivation at different stages of the project.

I also express my thanks to **Dr. A. Shobana Devi**, Associate Professor and Project Coordinator, and **Dr. Selvakumarasamy**, Associate Professor and Panel Head, Department of Computational Intelligence, SRM Institute of Science and Technology, for their guidance and support during the project reviews.

I gratefully acknowledge **Mr. Ben Kingsley**, Program Manager, Great Learning, for his support and guidance throughout the academic program.

My special thanks to my project guide, **Dr. Selvakumarasamy**, Associate Professor, Department of Computational Intelligence, SRM Institute of Science and Technology, for giving me the opportunity to work under his mentorship. His guidance, encouragement, and support greatly contributed to the successful completion of this project.

I also thank the faculty members, staff, and students of the Department of Computational Intelligence, SRM Institute of Science and Technology, for their cooperation and assistance.

Finally, I express my heartfelt gratitude to my wife, parents, family members, and friends for their constant encouragement and unwavering support throughout this journey.

Darish M [PA2312049010073]

TABLE OF CONTENTS

<i>C.NO.</i>	<i>TITLE</i>	<i>PAGE NO.</i>
	Abstract	viii
	List of Tables	ix
	List of Figures	x
	List of Symbols and Abbreviations	xi
1.	INTRODUCTION	1
1.1	General	1
1.2	Purpose	2
1.3	Scope	3
1.4	Artificial Intelligence, Machine Learning and Deep Learning	3
1.5	Retrieval Augmented Generation	4
2	LITERATURE REVIEW	6
2.1	Retrieval Augmented Generation	6
2.2	Retrieval Methods and Ranking	6
2.3	Multi-Lingual and Domain Adaptive Retrieval	7
2.4	Document Understanding and OCR	7
2.5	Hallucination and Grounded Generation	7
2.6	Research Gap and Motivation	8
3	PROPOSED METHODOLOGY	9
3.1	Dataset	9
3.2	Document Processing Layer	11
3.2.1	OCR and Text Extraction	12
3.2.2	Metadata Extraction	13
3.2.3	Clause-Aware Chunking	14
3.2.4	Embedding Generation	15
3.2.5	Indexing using FAISS	15
3.3	Hybrid Retrieval Layer	16
3.3.1	Query Representation	16

3.3.2	Dense Semantic Retrieval	16
3.3.3	Sparse Retrieval using BM25	17
3.3.4	Hybrid Retrieval Fusion (RRF)	17
3.3.5	Reranking	17
3.3.6	Multi-GO Reasoning	17
3.3.7	Temporal Prioritization	18
3.4	Generation and Reasoning Layer	18
3.4.1	Context Construction	20
3.4.2	Prompt Construction and Control	20
3.4.3	Response Generation	21
3.4.4	Multi-Document Reasoning	21
3.4.5	Citation Grounding	22
3.4.6	Hallucination Control	22
3.5	Metrics	23
3.5.1	Evaluation Benchmark for GOVRAG	23
3.5.2	Retrieval Metrics	23
3.5.3	Generation Metrics	25
3.6	Implementation Details	26
3.6.1	Overall algorithm	27
3.6.2	Step-wise Algorithm	27
3.6.3	Pseudocode Representation	29
4	RESULTS	32
4.1	Retrieval Comparison	32
4.2	Clause-level Retrieval Analysis	34
4.3	Generation Performance Evaluation	34
4.4	Hallucination Analysis	35
4.5	Impact of Multi-GO Reasoning	36
4.6	Temporal Policy Reasoning Evaluation	36
4.7	Discussion	36
5	CONCLUSION	39
6	FUTURE SCOPE	41
7	REFERENCES	42
	APPENDIX 1	46

APPENDIX 2

49

PAPER PUBLICATION STATUS

53

PLAGIARISM REPORT

54

ABSTRACT

Government Orders (GOs) serve as critical instruments for implementing administrative decisions, policy directives, and regulatory frameworks in Tamil Nadu, India. However, effective access to information within these documents remains challenging due to their unstructured formats, scanned documents, often captured using mobile devices, bilingual content (English and Tamil), and continuous amendments. Traditional keyword-based retrieval systems fail to capture semantic intent, while direct application of large language models often results in hallucinated outputs, limiting their reliability in governance contexts.

In this paper, we present **GOVRAG-TN**, a domain-adaptive cross-lingual Retrieval-Augmented Generation (RAG) architecture designed for the trustworthy interpretation of Tamil Nadu Government Orders. We have integrated bilingual optical character recognition, structure-aware clause-level chunking, multilingual semantic embeddings, and a hybrid retrieval framework combining dense and lexical search with reranking. While the conventional RAG systems operate on isolated documents, we in GOVRAG-TN introduce **Multi-GO Reasoning**, enabling aggregation of evidence across multiple Government Orders, and a **Temporal Policy Prioritization module**, which emphasizes recent amendments to ensure policy relevance.

We have conducted experiments on a corpus of over 9,000 scanned Government Orders of Tamil Nadu Government and demonstrated improvements in semantic retrieval and grounded response generation. Our system achieves a Recall@5 of 0.52 and significantly reduces hallucination rates while maintaining clause-level citation accuracy. We have designed the architecture in a way that it can be deployed fully offline using open-source models, making it suitable for governance environments with privacy and infrastructure constraints.

LIST OF TABLES

1	Retrieval Performance Across Methods	32
2	Generation Performance Across Methods	34

LIST OF FIGURES

1	Overall GOVRAG-TN architecture.....	5
2	Tamil Nadu Government GO repository.....	9
3	Department-wise repository.....	10
4	Sample Government Order document.....	10
5	Document Processing pipeline.....	11
6	OCR Input (PDF) vs Extracted Text Output.....	13
7	Example of clause-based segmentation from a GO	15
8	Hybrid Retrieval Architecture.....	16
9	Generation and Reasoning Pipeline.....	19
10	GOVRAG-TN User Interface.....	30
11	Generated Citation-Grounded Response.....	31
12	Retrieved Supporting Evidence.....	31
13	Key retrieval metrics.....	33
14	Retrieval performance comparison.....	33
15	GO Recall vs Clause Recall comparison.....	34
16	Performance heatmap.....	35
17	Hallucination rate comparison.....	36

LIST OF SYMBOLS AND ABBREVIATIONS

Symbol / Term	Meaning
GO	Government Order
RAG	Retrieval-Augmented Generation
OCR	Optical Character Recognition
BM25	Best Matching 25 (Ranking Algorithm)
FAISS	Facebook AI Similarity Search
RRF	Reciprocal Rank Fusion
MRR	Mean Reciprocal Rank
nDCG	Normalized Discounted Cumulative Gain
LM	Language Model
(q)	User Query
(D_i)	Document
(C_i)	Context
(e_j)	Embedding Vector
($TopK$)	Top-K Retrieved Results
NLP	Natural Language Processing
AI	Artificial Intelligence

CHAPTER 1

INTRODUCTION

1.1 General

Government Orders (GOs) constitute a foundational mechanism through which administrative decisions are formalized and communicated across departments. In the context of Indian governance, particularly within Tamil Nadu, these orders encapsulate a wide range of policy actions, including financial allocations, institutional regulations, amendments, and procedural directives. In principle, they serve as authoritative references for both policymakers and the public. In practice, however, accessing and interpreting these documents remains a non-trivial task.

A significant portion of Government Orders are disseminated as scanned PDF documents, often containing a mixture of Tamil and English text. This introduces multiple layers of complexity. First, the lack of machine-readable text limits direct computational processing. Second, the bilingual nature of the documents demands cross-lingual understanding. Third, the legal and administrative structure of GOs—characterized by clauses, references to prior orders, and layered amendments—makes them inherently difficult to parse using conventional methods. It is worth noting that these challenges are not merely technical inconveniences; they directly affect transparency, accessibility, and informed decision-making.

Traditional keyword-based retrieval systems, which rely heavily on exact term matching, are insufficient in this setting. Such systems implicitly assume that users are aware of specific document identifiers or terminology, which is rarely the case in real-world scenarios. As a result, users often struggle to retrieve relevant information, even when it exists within the corpus. More recently, natural language-based systems have attempted to address this limitation. However, when applied directly to governance documents, they introduce a different concern—namely, the generation of plausible yet factually incorrect responses.

Retrieval-Augmented Generation (RAG) has emerged as a promising paradigm to bridge this gap by combining information retrieval with response generation. By grounding generated outputs in external documents, RAG systems aim to improve

factual consistency. That said, most existing RAG implementations are designed for relatively unstructured or general-purpose corpora. When applied to Government Orders, these systems tend to overlook critical aspects such as document structure, inter-document dependencies, and temporal evolution of policies. This gap motivates the need for a more domain-sensitive approach.

1.2 Purpose

The primary objective of this research is to design and develop a domain-adaptive cross-lingual Retrieval-Augmented Generation framework, termed **GOVRAG-TN**, for the trustworthy interpretation of Tamil Nadu Government Orders. The system is intended not merely as a retrieval tool, but as an end-to-end architecture capable of transforming unstructured, multilingual, and scanned documents into interpretable and verifiable responses.

A key motivation behind this work is the observation that existing approaches treat governance documents in a largely generic manner. This often results in a loss of contextual fidelity, particularly when dealing with amendments, cross-references, and evolving policy narratives. GOVRAG-TN attempts to address this by incorporating structure-aware processing mechanisms that preserve the legal hierarchy and semantic relationships within documents.

Another important aspect is cross-lingual accessibility. Given that Government Orders frequently contain both Tamil and English text, the system is designed to operate seamlessly across languages, enabling users to query in one language and retrieve information from documents written in another. This is particularly relevant in a diverse administrative setting where linguistic barriers can limit access to critical information.

In addition, the framework emphasizes **trustworthiness**. Rather than generating free-form responses, the system produces citation-grounded outputs, allowing users to trace each piece of information back to its source. This is complemented by a multi-document reasoning capability, where evidence is aggregated across multiple Government Orders. Interestingly, this becomes crucial in cases where a policy is not defined within a single document but evolves over a series of amendments.

Finally, the system is designed with practical deployment constraints in mind. By leveraging open-source components and enabling offline operation, GOVRAG-TN

aims to be applicable within governance environments where data privacy and infrastructure limitations are significant considerations.

1.3 Scope

The scope of this study is centered on the design, implementation, and evaluation of a domain-specific RAG architecture tailored for Tamil Nadu Government Orders. The dataset comprises a large corpus of over 9,000 scanned GOs obtained from the official government repository. These documents serve as the primary knowledge base for all retrieval and generation tasks.

The work encompasses multiple stages of processing, beginning with optical character recognition (OCR) for bilingual text extraction, followed by structure-aware document segmentation. Unlike conventional approaches that rely on fixed-size chunking, this system considers legal and semantic boundaries within documents, such as clauses and sections. This distinction, although subtle, has a noticeable impact on retrieval quality.

On the retrieval side, the framework explores hybrid strategies that combine lexical and dense representations. This allows the system to balance exact term matching with semantic similarity, thereby improving recall without sacrificing precision. The retrieved content is then passed to a generation module that produces responses grounded in the selected evidence, with explicit citations.

It should be noted that the scope of this work is limited to Government Orders within Tamil Nadu and does not directly extend to other forms of legal or administrative documents. However, the architectural principles and design choices are generalizable and could potentially be adapted to similar domains with minimal modifications.

1.4 Artificial Intelligence, Machine Learning and Deep Learning

The rapid advancement of Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL) has significantly transformed the way unstructured data is processed and interpreted. In recent years, these technologies have been increasingly applied to tasks involving natural language understanding, document analysis, and information retrieval.

Machine Learning techniques, particularly those based on statistical learning, have traditionally been used for tasks such as classification and clustering. However, their effectiveness is often limited by the need for handcrafted features and domain-specific

tuning. Deep Learning models, on the other hand, have demonstrated a remarkable ability to learn hierarchical representations directly from data. This has enabled more robust handling of complex language patterns and semantic relationships.

In the context of document processing, deep neural architectures have been instrumental in tasks such as optical character recognition, language modeling, and semantic embedding generation. These capabilities form the backbone of modern retrieval systems, allowing them to move beyond surface-level keyword matching. That said, it is important to recognize that these models are not inherently designed for high-stakes domains such as governance. Without proper grounding mechanisms, they may produce outputs that are coherent but not necessarily accurate.

This limitation highlights the importance of integrating retrieval mechanisms with generative models, as seen in RAG-based systems. By anchoring generated responses to actual documents, such systems attempt to balance fluency with factual correctness. However, as discussed earlier, domain adaptation remains a critical requirement—particularly when dealing with structured and evolving documents like Government Orders.

1.5 Retrieval Augmented Generation

The GOVRAG-TN framework is designed as a modular Retrieval-Augmented Generation (RAG) system for the reliable interpretation of Tamil Nadu Government Orders. The architecture integrates document preprocessing, hybrid retrieval, and citation-grounded generation within a unified pipeline. It addresses key challenges such as multilingual content, scanned document formats, preservation of legal structure, and temporal policy dependencies. As illustrated in Fig. 1, the system is organized into three primary layers: the Document Processing Layer, the Hybrid Retrieval Layer, and the Generation and Reasoning Layer. These layers operate sequentially, transforming raw Government Order documents into grounded and verifiable responses.

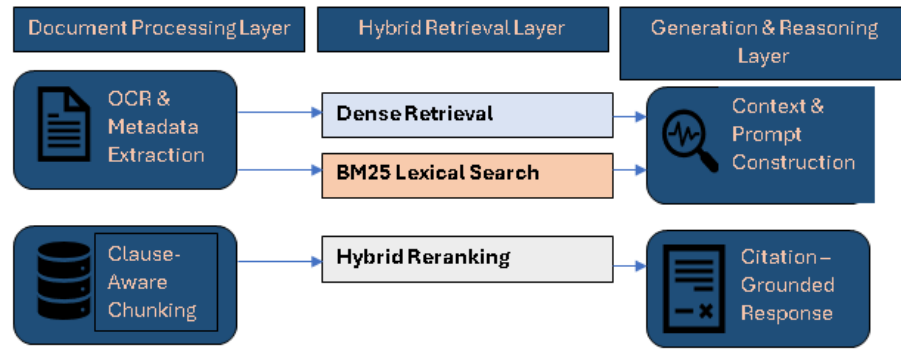


Fig. 1. Overall GOVRAG-TN architecture

CHAPTER 2

LITERATURE REVIEW

The study of retrieval and interpretation of complex documents has evolved significantly with the advancement of information retrieval and natural language processing techniques. In the context of governance documents, the challenge is not limited to retrieving relevant text but extends to ensuring semantic correctness, interpretability, and traceability of information. This section reviews the major developments relevant to retrieval-augmented systems, multilingual retrieval, document understanding, and grounded generation, while identifying the gaps that motivate the proposed GOVRAG-TN framework.

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation has emerged as a key paradigm for improving factual grounding in language models by integrating external knowledge retrieval with generative models. Early work demonstrated that augmenting generation with retrieved documents significantly enhances performance in knowledge-intensive tasks [1]. Subsequent approaches showed that leveraging multiple retrieved passages improves reasoning across sources and reduces reliance on parametric knowledge [3]. Retrieval-augmented methods have since become central to building trustworthy and interpretable language systems, particularly in domains requiring high factual accuracy.

2.2 Retrieval Methods and Ranking

Classical information retrieval systems rely on lexical matching approaches such as BM25, which remain strong baselines for keyword-based search [45]. However, these methods are limited in capturing semantic relationships. Dense retrieval approaches address this limitation by representing queries and documents in a shared embedding space, enabling semantic similarity matching [2]. Advances in representation learning, including Sentence-BERT [6] and transformer-based models such as BERT and RoBERTa [30, 14], have significantly improved retrieval performance.

Ranking plays a critical role in retrieval pipelines. Cross-encoder models improve relevance estimation by jointly encoding query–document pairs [21], while efficient

reranking approaches aim to balance accuracy and computational cost [22]. Interaction-based models such as ColBERT introduce late interaction mechanisms that enable scalable yet effective retrieval [41]. Additionally, developments in approximate nearest neighbor search and indexing techniques, including FAISS, have made large-scale dense retrieval practical [44].

2.3 Multilingual and Domain-Adaptive Retrieval

Multilingual retrieval introduces additional complexity, particularly in datasets containing mixed-language content. Cross-lingual models such as XLM-R provide strong baselines for multilingual understanding [10], while contrastive embedding approaches improve semantic alignment across languages [11]. Domain adaptation further enhances performance in specialized domains, with models such as LegalBERT demonstrating improvements in legal text understanding [7, 8].

However, retrieval effectiveness is influenced not only by embeddings but also by document structure. Studies have shown that preserving document structure significantly improves retrieval performance in hierarchical texts [13]. This is particularly important in governance documents, where meaning is often tied to clauses, sections, and amendments.

2.4 Document Understanding and OCR

A significant portion of government data exists in scanned formats, necessitating robust Optical Character Recognition (OCR) systems. Early work highlighted challenges in scene text recognition [15], while modern systems such as PaddleOCR provide improved robustness for multilingual document extraction [16]. Beyond text extraction, document understanding has evolved through layout-aware models such as LayoutLM and its variants [17–19], as well as multimodal transformer architectures that integrate textual and spatial features [20].

These approaches are particularly relevant for structured documents where layout and formatting influence meaning, such as Government Orders.

2.5 Hallucination and Grounded Generation

Despite advances in generative models, hallucination remains a critical issue. Large language models are prone to generating unsupported or fabricated information, particularly in domain-sensitive applications [23]. Retrieval-based grounding has been

shown to mitigate hallucination and improve factual consistency [24]. This has led to increased interest in combining retrieval with generation to produce verifiable and explainable outputs.

2.6 Research Gap and Motivation

Despite the significant progress in retrieval-augmented systems, several limitations remain when these approaches are applied to governance documents.

First, most existing systems assume unstructured text, whereas Government Orders exhibit semi-structured formats with legal hierarchies. Second, cross-lingual retrieval remains a challenge, particularly when domain-specific terminology is involved. Third, conventional chunking strategies fail to preserve clause-level dependencies, leading to loss of contextual information. Finally, temporal reasoning—understanding how policies evolve over time—is largely overlooked.

These gaps collectively suggest that a domain-adaptive approach is necessary. The proposed GOVRAG-TN framework addresses these challenges by integrating structure-aware chunking, multilingual retrieval, hybrid search, multi-document reasoning, and citation-grounded generation into a unified architecture.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Dataset

The dataset used in this study consists of Tamil Nadu Government Orders (GOs) collected from the official government portal (*tn.gov.in*). The corpus spans around than 9,000 documents across 38 departments, as shown in Fig.2, including finance, health, agriculture, human resources, and special programs. As shown in Fig.3, each departments' GO are stored in the website.

Unlike standard NLP datasets, the collected documents present several real-world challenges:

- Documents are predominantly scanned PDF files
- Content is bilingual (Tamil and English)
- Documents follow a semi-structured administrative format
- Presence of policy evolution through amendments and references

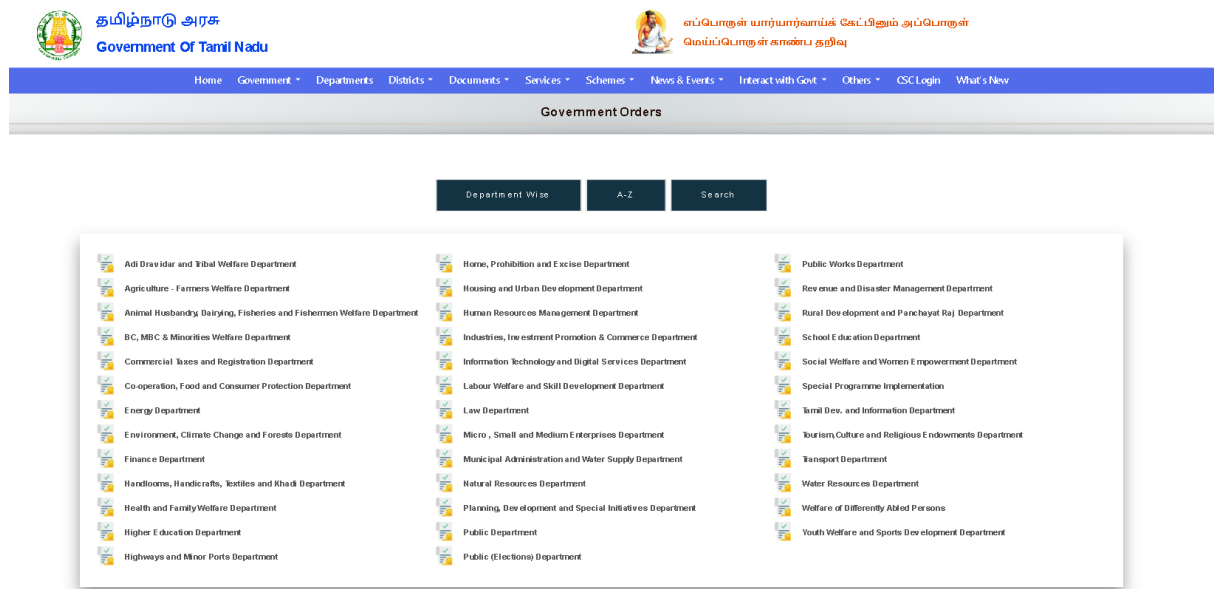


Fig. 2. Tamil Nadu Government GO repository

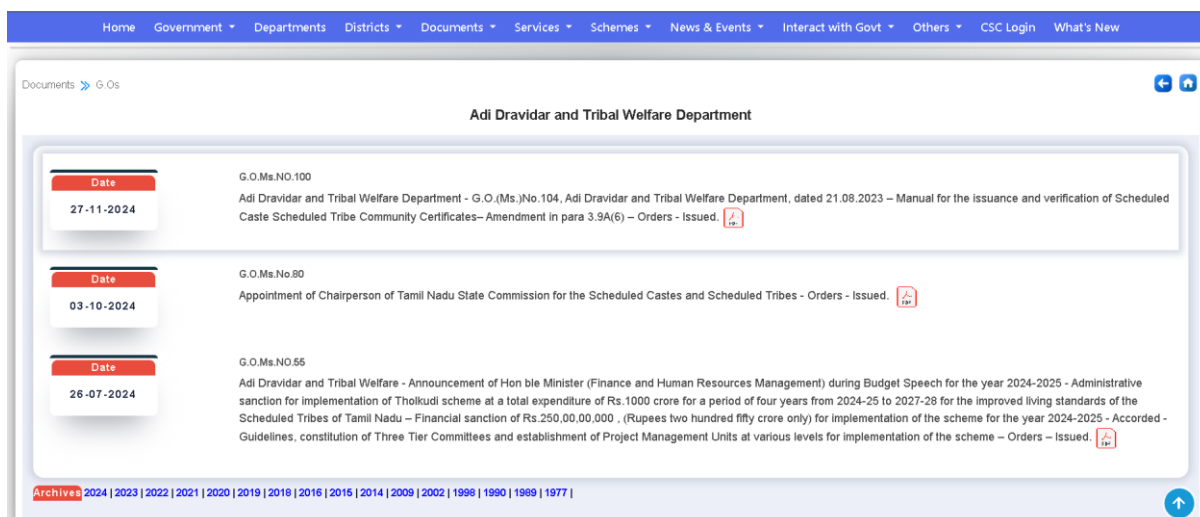


Fig. 3. Department-wise repository

As shown in Fig. 4, each document typically contains structured components such as:

- Abstract
- Government Order references
- Clauses and sub-clauses
- Annexures and tabular data

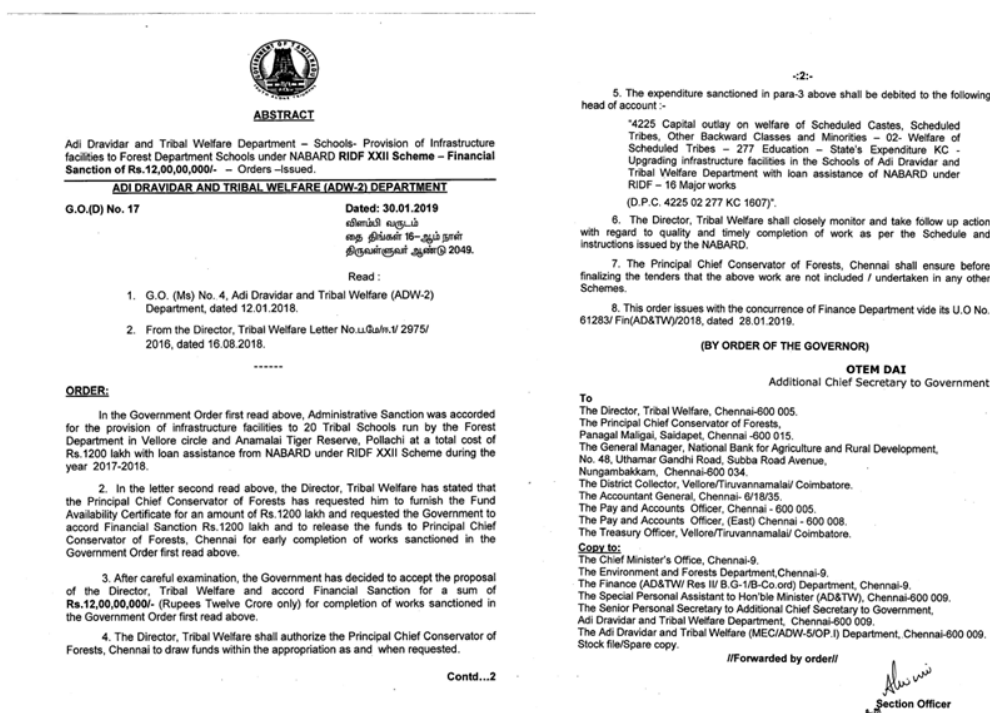


Fig. 4. Sample Government Order document

3.2 Document Processing Layer

The Document Processing Layer prepares Government Order documents for semantic retrieval. As illustrated in Fig. 5, the input consists of official Government Orders collected from the Tamil Nadu government portal, which are predominantly available as scanned or digitally generated PDF files containing multilingual content and structured administrative text.

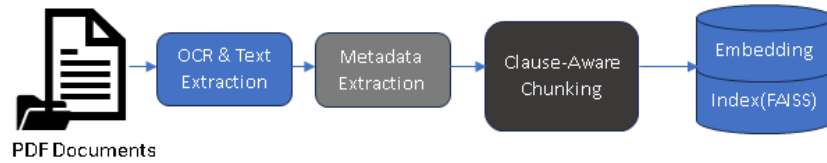


Fig. 5. Document processing pipeline

The processing begins with digitization through Optical Character Recognition (OCR), where PDF documents are converted into machine-readable text. This step is particularly critical for scanned documents where content is stored as images. To ensure a fully offline pipeline, Tesseract OCR is employed. The extracted text is subsequently normalized to remove noise, correct encoding inconsistencies, and preserve the original document structure, including headings and clause boundaries.

Following text extraction, structured metadata is identified and associated with each document. This includes Government Order number, department name, year of publication, section identifiers, and clause numbers. Retaining this metadata throughout the pipeline is essential for enabling citation-grounded response generation in later stages.

A key component of this layer is clause-aware chunking, which segments documents based on their inherent legal structure. Government Orders typically consist of abstracts, clauses, sub-clauses, and annexures, and this hierarchy is preserved during segmentation. Formally, a document D is decomposed into a set of clause-level units:

$$D = \{c_1, c_2, \dots, c_n\}$$

where each chunk c_i represents a semantically coherent clause along with its identifiers and associated metadata. This approach ensures that embeddings correspond to

meaningful legal units rather than arbitrary text segments, thereby improving retrieval accuracy.

Each chunk is then encoded into a dense vector representation using the multilingual-e5-large embedding model, which is selected for its strong cross-lingual retrieval capabilities. The embedding of a clause c_i is defined as:

$$e_i = f_{embed}(c_i)$$

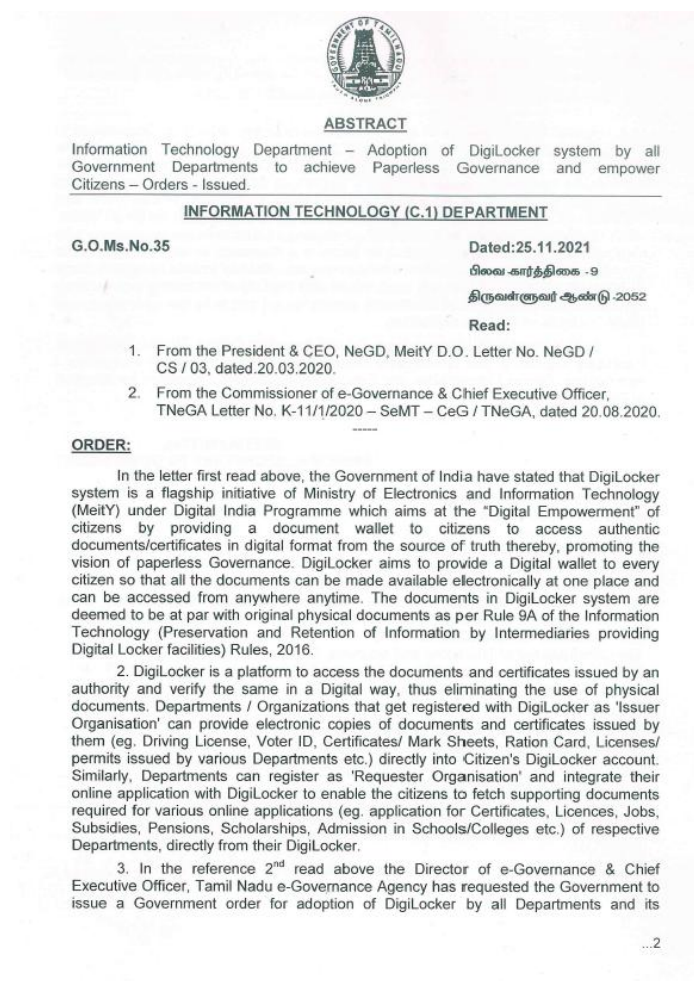
where $f_{embed}(\cdot)$ denotes the embedding function. The resulting embeddings are indexed using FAISS, enabling efficient similarity search over large-scale document collections in a fully offline environment.

3.2.1 OCR and Text Extraction

Since a large portion of the Government Orders are available as scanned PDFs, the first step involves converting them into machine-readable text using Optical Character Recognition (OCR) as shown in Fig. 6.

The OCR process attempts to preserve both textual content and layout structure. However, due to variations in scan quality, font styles, and bilingual content, the extracted text may contain noise such as:

- Character misrecognition
- Broken words or merged tokens
- Inconsistent spacing



(a)

```

1  {
2    "source_pdf": "it_e_35_2021_ms.pdf",
3    "department_folder": "information technology and digital services department",
4    "pages": [
5      {
6        "page_no": 1,
7        "text": "ABSTRACT\nInformation Technology Department - Adoption of DigiLocker system by all\nGovernment Departments to achieve Paperless",
8      },
9      {
10       "page_no": 2,
11       "text": "=\nagencies towards supporting Paperless Governance. The Government of India have\nalso requested to adopt DigiLocker as a pla",
12     }
13   ]
14 }

```

(b)

Fig. 6. (a) OCR Input (PDF) vs (b) Extracted Text Output

3.2.2 Metadata Extraction

The system extracts key metadata elements that are essential for both retrieval and interpretability.

The extraction process relies on pattern recognition techniques and heuristic rules tailored to Government Order formats. For instance:

- GO numbers often follow identifiable numbering schemes
- Department names appear in standardized headers
- Dates and years are typically present in structured formats

This metadata serves multiple purposes:

- Acts as filtering signals during retrieval
- Enables citation grounding in generated responses
- Supports temporal reasoning by distinguishing between older and newer policies

Interestingly, even when textual content is ambiguous, metadata often provides strong contextual anchors for interpretation.

3.2.3 Clause-Aware Chunking

A critical design decision in this work is the adoption of clause-aware chunking, as opposed to fixed-length segmentation. Traditional chunking strategies divide text into equal-sized segments, which can disrupt semantic continuity—particularly problematic in legal or administrative documents.

This approach ensures that each chunk corresponds to a semantically complete unit, preserving:

- Legal meaning
- Contextual dependencies
- Referential links to other clauses

It is worth noting that Government Orders often include nested structures (e.g., sub-clauses, annexures). While fully modeling hierarchical dependencies is complex, clause-level segmentation provides a practical balance between granularity and interpretability. An example of clause-based chunking is shown in Fig. 7.

```
[
{
  "text": "G.O(Ms)No.215, Law (LS) Department,dated.02.04.2012.",
  "metadata": {
    "abstract": "",
    "department": "Law Department",
    "go_number": "G.O.(Ms.) No. 181",
    "date": "26.12.2014",
    "year": "2014",
    "preceding_go_numbers": [
      "G.O(Ms)No.215",
      "G.O.(Ms.) No. 181"
    ],
    "to": "",
    "copy_to": "1. The Director Legal Studies, Kilpauk, Chennai-600 010.\n2.",
    "annexure_presence": false,
    "chunk_type": "clause",
    "section_id": "1"
  }
}
```

Fig. 7. Example of clause-based segmentation from a GO

3.2.4 Embedding Generation

Each chunk is transformed into a dense vector representation using a multilingual embedding model

The choice of embedding model is particularly important in this context. Since the dataset contains bilingual content, the embedding space must support cross-lingual semantic alignment, allowing queries in one language to retrieve relevant content in another.

These embeddings capture semantic relationships beyond surface-level text matching, enabling the system to handle paraphrased queries and varied linguistic expressions.

3.2.5 Indexing using FAISS

To enable efficient retrieval over a large corpus, embeddings are indexed using FAISS (Facebook AI Similarity Search).

FAISS provides:

- Sub-linear search complexity
- Scalability for large datasets
- Compatibility with offline deployment

This is particularly relevant in governance settings where:

- Internet access may be restricted
- Data privacy constraints prohibit external API usage

3.3 Hybrid Retrieval Layer

The Hybrid Retrieval Layer is responsible for identifying the most relevant Government Order clauses corresponding to a user query. GOVRAG employs a hybrid retrieval strategy as shown in Fig. 8, that combines dense semantic retrieval with lexical search to improve both recall and precision.

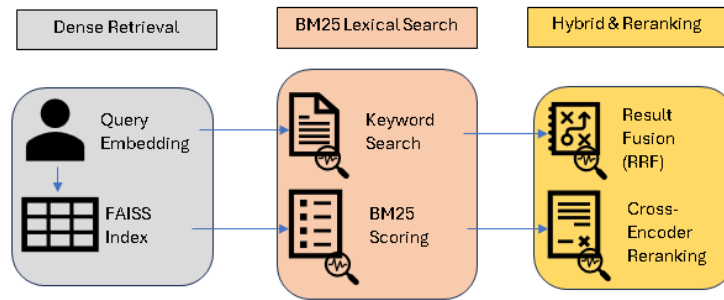


Fig. 8. Hybrid retrieval architecture

3.3.1 Query Representation

Given a query q , a dense embedding is first generated using the same multilingual model:

$$e_q = f_{embed}(q)$$

3.3.2 Dense Semantic Retrieval

Dense semantic retrieval is performed using similarity search over the FAISS index, enabling the system to capture semantic relationships even when keyword overlap is minimal. The similarity between the query and a clause c_i is computed using cosine similarity:

$$sim(q, c_i) = \frac{e_q \cdot e_i}{||e_q|| ||e_i||}$$

3.3.3 Sparse Retrieval using BM25

To complement semantic retrieval, lexical retrieval is performed using BM25, which captures term-based relevance between the query and document text. The BM25 scoring function is given by:

$$BM25(q, c_i) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, c_i)(k_1 + 1)}{f(t, c_i) + k_1(1 - b + b \cdot \frac{|c_i|}{avgdl})}$$

where $f(t, c_i)$ represents term frequency and k_1, b are hyperparameters.

3.3.4 Hybrid Retrieval Fusion (RRF)

The outputs of dense and lexical retrieval are combined using Reciprocal Rank Fusion (RRF), which aggregates rankings from multiple retrieval methods:

$$RRF(c_i) = \sum_{j=1}^n \frac{1}{k + rank_j(c_i)}$$

where $rank_j(c_i)$ denotes the rank of clause c_i in the j^{th} retrieval method and k is a constant, typically set to 60. This fusion strategy preserves the strengths of both retrieval approaches and improves robustness.

3.3.5 Reranking

To further refine relevance, a cross-encoder reranker is applied to the top retrieved candidates. The reranker jointly encodes the query–document pair and produces a fine-grained relevance score:

$$s_i = f_{rerank}(q, c_i)$$

This multi-stage retrieval pipeline improves both recall and precision for policy-oriented queries.

3.3.6 Multi-GO Reasoning

A notable limitation of conventional retrieval systems is their reliance on single-document evidence. In contrast, Government Orders often distribute relevant information across multiple documents, especially when policies evolve over time.

To address this, the system aggregates information across retrieved documents:

$$C_q = \bigcup_{d \in TopK'} d$$

This enables:

- Cross-document evidence integration
- Linking of related policies
- Reconstruction of policy evolution

It is worth noting that this aggregation is not purely additive; relevance signals are preserved to ensure that more important documents contribute proportionally to the final context.

3.3.7 Temporal Prioritization

Government policies are inherently dynamic. Newer Government Orders may override or modify earlier ones. Therefore, the system incorporates a temporal weighting mechanism.

For a document d with publication year $Year_d$, the temporal weight is defined as:

$$w(d) = \frac{1}{1 + \lambda(Current\ Year - Year_d)}$$

where:

- λ controls the rate of decay

The adjusted score becomes:

$$score'(d) = score(d) \cdot w(d)$$

This ensures that:

- Recent documents are prioritized
- Outdated policies have reduced influence

3.4 Generation and Reasoning Layer

The Generation and Reasoning Layer produces natural language responses grounded in the retrieved policy documents. As illustrated in Fig. 9, this layer integrates context construction, prompt engineering, and language model inference to ensure accurate and interpretable outputs.

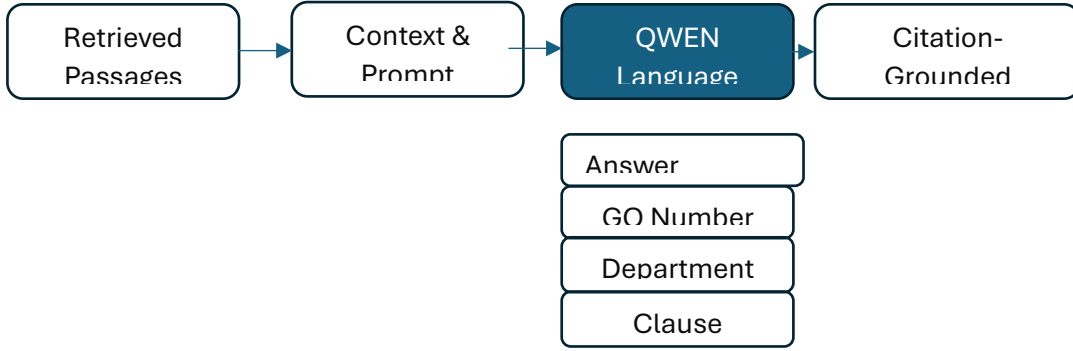


Fig. 9. Generation and Reasoning pipeline

The top-ranked clauses are aggregated to form a structured context window:

$$\mathcal{C} = \{c_{i_1}, c_{i_2}, \dots, c_{i_k}\}$$

Each context element retains metadata such as Government Order number, department, year, abstract, and clause identifiers. This ensures that the language model receives both content and provenance information.

A structured prompt is then constructed to guide the language model. The prompt explicitly constrains the model to generate responses based only on the retrieved evidence and to include relevant Government Order numbers and clause references. This constraint-driven prompting reduces hallucination and enhances transparency.

The final response is generated using a locally deployed large language model:

$$y = f_{LLM}(q, \mathcal{C})$$

where y represents the generated output. GOVRAG utilizes Qwen2.5 (7B) deployed via the Ollama framework, enabling fully offline inference on CPU-based systems and ensuring compliance with data privacy requirements.

Each generated response includes explicit references to Government Order number, department, year, and clause identifiers. This citation-grounded output enables users to verify responses against the original documents, ensuring explainability and trustworthiness. The architecture further supports multi-document reasoning by aggregating evidence across multiple Government Orders and incorporates temporal prioritization to ensure alignment with the most recent policy updates.

3.4.1 Context Construction

The first step involves constructing a context window from the top-ranked retrieved clauses. Given the refined retrieval set $TopK'$, the context is defined as:

$$C = \bigcup_{d \in TopK'} d$$

Each context element d retains its associated metadata:

$$d = (text_d, GO_id_d, clause_id_d, Dept_d, Year_d)$$

This ensures that the context is not merely a collection of text fragments but a structured representation of policy evidence.

An important consideration here is **context size limitation**. Since language models operate within fixed input lengths, only the most relevant chunks are selected. This introduces a trade-off: including more context may improve coverage but can dilute relevance or exceed computational constraints.

3.4.2 Prompt Construction and Control

To guide the generation process, a structured prompt template is employed. The prompt can be represented as:

$$P = Template(q, C)$$

where:

- q is the user query
- C is the constructed context

The prompt explicitly instructs the model to:

- Base the response strictly on provided context
- Avoid introducing external or assumed information
- Include references to Government Orders and clauses

This controlled prompting acts as a soft constraint mechanism. While it does not guarantee complete elimination of unsupported outputs, it significantly reduces the likelihood of hallucination.

It is worth noting that prompt design plays a non-trivial role. Minor variations in instruction phrasing can influence how strictly the model adheres to the provided evidence.

3.4.3 Response Generation

The final response is generated using a locally deployed language model:

$$y = LM(P)$$

where:

- LM corresponds to **Qwen2.5 (7B)**
- P is the structured prompt

The choice of a locally deployed model is motivated by:

- Data privacy requirements in governance settings
- Reduced dependency on external APIs
- Greater control over system behavior

Unlike large-scale cloud-based systems, local deployment introduces constraints in terms of computational resources. However, it provides a more practical and secure solution for administrative use cases.

3.4.4 Multi-Document Reasoning

An important capability of this layer is its ability to synthesize information across multiple documents. Since the context CCC may include clauses from different Government Orders, the model must implicitly perform reasoning over this aggregated information.

Formally, the generated response can be viewed as:

$$y = f_{reason}(C, q)$$

where f_{reason} represents the model's ability to:

- Identify relevant relationships between clauses
- Resolve partial overlaps in information
- Construct a unified interpretation

This becomes particularly important in scenarios where:

- Policies are defined across multiple GOs
- Amendments modify earlier provisions
- Information is distributed rather than centralized

Interestingly, while the reasoning is not explicitly programmed, the structure of the context and prompt strongly influences the model’s ability to perform such aggregation.

3.4.5 Citation Grounding

To ensure transparency and verifiability, each generated response is augmented with explicit references. The output structure is defined as:

$$y = \{answer, GO_id, clause_id, Dept, Year\}$$

This allows users to:

- Trace each statement back to its source
- Verify correctness against official documents
- Understand the origin of policy information

Citation grounding also serves as an implicit validation mechanism. If a response cannot be supported by retrieved evidence, it is more likely to be flagged as incomplete or uncertain.

3.4.6 Hallucination Control

Hallucination, defined as the generation of unsupported or fabricated information, is a known limitation of generative systems. In this framework, hallucination is controlled through a combination of:

1. **Retrieval grounding** (restricting input to verified context)
2. **Prompt constraints** (explicit instructions to avoid unsupported claims)
3. **Citation enforcement** (requiring traceable outputs)

The hallucination rate can be quantified as:

$$Hallucination\ Rate = \frac{Number\ of\ unsupported\ responses}{Total\ responses}$$

While these mechanisms significantly reduce hallucinations, they do not eliminate them entirely. A potential limitation arises when the retrieved context is incomplete; in such cases, the model may still attempt to generate plausible but partially unsupported responses.

3.5 Metrics

3.5.1 Evaluation Benchmark for GOVRAG

A benchmark of 50 manually created question–answer pairs is used to assess the system performance. The queries are designed to reflect realistic user requirements such as factual queries (e.g., beneficiaries), procedural queries (e.g., rules), and boolean queries (e.g., presence of provisions).

Each query is related with the corresponding Government Order, relevant clauses, and expected answer. This ground-truth dataset helps in the evaluation of both retrieval and generation performance.

3.5.2 Retrieval Metrics

Recall@K evaluates whether the retrieval system is capable of identifying relevant Government Order clauses within the top-ranked search results. Since the downstream generation stage depends heavily on retrieved evidence, higher recall generally leads to better contextual coverage. In the context of GOVRAG-TN, this metric is particularly important because policy-related information is often distributed across multiple clauses or documents.

$$Recall@K = \frac{|Rel \cap TopK|}{|Rel|}$$

where Rel denotes relevant documents for a query, and $TopK$ represents the top-K retrieved documents. A higher Recall@K value indicates that the system is effective in capturing relevant information within the top-ranked results, which is particularly important in retrieval-augmented generation systems where downstream performance depends on the availability of relevant context.

Precision@K measures the proportion of retrieved results that are actually relevant to the query. While recall focuses on coverage, precision reflects retrieval quality by minimizing unrelated clauses in the final context. This becomes important in

governance applications, where inclusion of irrelevant policy text can negatively influence response generation.

$$Precision@K = \frac{|Rel \cap TopK|}{K}$$

where Rel represents the relevant documents for a query, and $TopK$ represents the top- K retrieved documents. A higher $Precision@K$ indicates that the system retrieves fewer irrelevant documents, thereby improving the quality of the context provided for downstream generation tasks.

MRR evaluates how early the first relevant Government Order appears in the ranked retrieval results. In real-world usage, users typically rely on the top few retrieved passages rather than scanning long lists of documents. Therefore, retrieving relevant evidence at earlier ranks directly improves system usability and response quality.

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

where Q denotes the set of queries, $rank_i$ represents the rank position of the first relevant document for the i^{th} query. A higher MRR value indicates that relevant documents are retrieved earlier in the ranking, which is critical for systems where only top-ranked results are used for downstream tasks.

Normalized Discounted Cumulative Gain (nDCG@10) evaluates ranking effectiveness while accounting for the position of relevant documents in the retrieval list. The metric gives greater importance to highly relevant clauses appearing at higher ranks, making it suitable for evaluating retrieval systems where ranking order significantly affects downstream reasoning.

$$nDCG@10 = \frac{DCG@10}{IDCG@10}$$

where,

$$DCG@10 = \sum_{i=1}^{10} \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

and rel_i denotes the graded relevance of the document at position i . The term $IDCG@10$ represents the maximum possible DCG obtained by an ideal ranking. A

higher $nDCG@10$ value indicates better ranking quality, particularly in placing highly relevant documents at top positions.

GO Recall evaluates whether the correct Government Order is successfully retrieved for a given query. Unlike standard retrieval metrics that focus only on document relevance, this metric specifically measures source-level correctness, which is important in administrative and policy-oriented systems.

$$GO\ Recall = \frac{\text{Number of queries with correct GO retrieved}}{\text{Total queries}}$$

Clause Recall measures the ability of the retrieval system to correctly identify the specific clause relevant to a given query. Unlike Government Order-level retrieval, which focuses on identifying the correct document, Clause Recall evaluates fine-grained retrieval at the clause level, where the exact answer is typically located. Formally, Clause Recall is defined as:

$$Clause\ Recall = \frac{\text{Number of queries with correct clause retrieved}}{\text{Total queries}}$$

3.5.3 Generation Metrics

Generation performance is evaluated using a set of accuracy and grounding-based metrics that assess the correctness, reliability, and interpretability of the generated responses.

Answer Accuracy measures how closely the generated response aligns with the expected ground truth answer for a given query. Since GOVRAG-TN operates in a policy interpretation setting, correctness of the generated explanation is more important than linguistic fluency alone.

$$Answer\ Accuracy = \frac{\text{Correct answers}}{\text{Total queries}}$$

Clause Citation Accuracy evaluates the correctness of clause-level references included in the generated responses. Since GOVRAG aims to produce citation-grounded outputs, this metric assesses whether the cited clauses correspond to the actual source of information. It is defined as,

$$Clause\ Citation\ Accuracy = \frac{\text{Correct clause citations}}{\text{Total responses}}$$

Government Order Mention Accuracy measures the correctness of identifying the relevant Government Order in the generated response. This metric ensures that the system retrieves and references the appropriate source document. It is computed as,

$$GO\ Mention\ Accuracy = \frac{Correct\ GO\ mentions}{Total\ responses}$$

Hallucination Rate measures the proportion of generated responses that contain unsupported or fabricated information not grounded in the retrieved evidence. It is a critical metric for evaluating the reliability of generative systems in governance applications. Formally, it is defined as,

$$Hallucination\ Rate = \frac{Unsupported\ responses}{Total\ responses}$$

Boolean Accuracy evaluates the correctness of responses for binary (yes/no) queries, which are common in policy interpretation tasks. It is defined as,

$$Boolean\ Accuracy = \frac{Correct\ yes/no\ predictions}{Total\ boolean\ queries}$$

To bring linguistic variation into account, answer evaluation uses fuzzy similarity matching. This step allows tolerance to paraphrased responses.

3.6 Implementation Details

The GOVRAG framework integrates document processing, retrieval, and generation components within a unified pipeline. Clause-level embeddings are generated using the multilingual-e5-large model, and FAISS is used for vector indexing. Lexical retrieval is performed using BM25, while a cross-encoder reranker based on MiniLM-L-6-v2 refines retrieval outputs. Response generation is handled by the Qwen2.5 (7B) language model deployed locally using Ollama, enabling fully offline operation.

Multiple retrieval strategies are evaluated, including dense semantic retrieval, dense retrieval with reranking, BM25-based lexical retrieval, hybrid retrieval combining semantic and lexical signals using Reciprocal Rank Fusion, and hybrid retrieval with reranking. These configurations enable systematic comparison across retrieval paradigms.

For each query q_i , the system retrieves top-K candidate clauses, optionally applies reranking, constructs a context window, and generates a response:

$$TopK = Retrieve(q_i), TopK' = Rerank(TopK, q_i)$$

$$C_i = \bigcup_{d \in TopK'} d, \quad y_i = LM(C_i, q_i)$$

All configurations are evaluated under identical conditions to ensure fair comparison.

3.6.1 Overall Algorithm

The GOVRAG-TN framework operates as a multi-stage pipeline that transforms raw Government Order documents into **grounded, interpretable responses**. While earlier sections described each component in isolation, this section presents the **end-to-end workflow**, highlighting how these components interact in a cohesive manner.

At a high level, the system follows four sequential stages:

1. Document ingestion and preprocessing
2. Structured representation and indexing
3. Hybrid retrieval and reasoning
4. Grounded response generation

These stages collectively enable the system to move from **unstructured scanned documents to verifiable answers**.

3.6.2 Step-wise Algorithm

The overall process can be described through the following sequence of operations:

Step 1: Document Collection and Preprocessing

Collect Government Order PDFs from the official repository

Apply OCR to convert scanned documents into text:

$$T_i = OCR(D_i)$$

Normalize extracted text:

$$T'_i = Normalize(T_i)$$

Step 2: Metadata Extraction and Chunking

Extract metadata:

$$M_i = \text{ExtractMeta}(T'_i)$$

Segment documents into clause-level chunks:

$$C_i = \{c_1, c_2, \dots, c_n\}$$

Step 3: Embedding and Indexing

Generate embeddings:

$$e_j = f_{\text{embed}}(c_j)$$

Store embeddings in FAISS index:

$$\text{Index} = \text{FAISS}(\{e_j\})$$

Step 4: Query Processing

Encode user query:

$$q_e = f_{\text{embed}}(q_i)$$

Step 5: Hybrid Retrieval

Perform dense retrieval:

$$\text{TopK}_{\text{dense}} = \text{Retrieve}_{\text{dense}}(q_e)$$

Perform sparse retrieval:

$$\text{TopK}_{\text{bm25}} = \text{Retrieve}_{\text{BM25}}(q_i)$$

Combine results using RRF:

$$\text{TopK}_i = \text{RRF}(\text{TopK}_{\text{dense}}, \text{TopK}_{\text{bm25}})$$

Step 6: Reranking

Refine results using cross-encoder:

$$\text{TopK}'_i = \text{Rerank}(\text{TopK}_i, q_i)$$

Step 7: Multi-GO Reasoning and Temporal Adjustment

Aggregate context:

$$Ci = \bigcup_{d \in \text{TopK}'_i} d$$

Apply temporal weighting:

$$score'(d) = score(d) \cdot w(d)$$

Step 8: Context Construction

Select top-ranked clauses with metadata

Construct structured context for generation

Step 9: Response Generation

Generate response:

$$y_i = LM(C_i, q_i)$$

Step 10: Citation Grounding

Attach:

GO number

Clause reference

Department

Year

3.6.3 Pseudocode Representation

Input: Query q

Output: Grounded Response y

1. $D \leftarrow$ Load Government Order PDFs
2. $T \leftarrow$ OCR (D)
3. $T' \leftarrow$ Normalize (T)
4. $M \leftarrow$ Extract Metadata (T')
5. $C \leftarrow$ Clause-aware Chunking (T')
6. $E \leftarrow$ Generate Embeddings (C)
7. $Index \leftarrow$ Build FAISS (E)
8. $q_e \leftarrow$ Encode (q)

9. Dense Results $\leftarrow$ Retrieve dense (q_e)
10. Sparse Results $\leftarrow$ Retrieve_BM25 (q)
11. Hybrid Results $\leftarrow$ RRF (Dense Results, Sparse Results)
12. Ranked Results $\leftarrow$ Rerank (Hybrid Results, q)
13. Context $\leftarrow$ Aggregate (Ranked Results)
14. Context $\leftarrow$ Apply Temporal Prioritization (Context)
15. $y \leftarrow$ Generate(Context, q)
16. $y \leftarrow$ Attach Citations(y)

Return y

The practical implementation of the GOVRAG-TN framework is illustrated in Figures 10 to 12. The appendix includes screenshots of the graphical user interface, sample query execution, generated responses, and retrieved evidence clauses demonstrating citation-grounded generation.

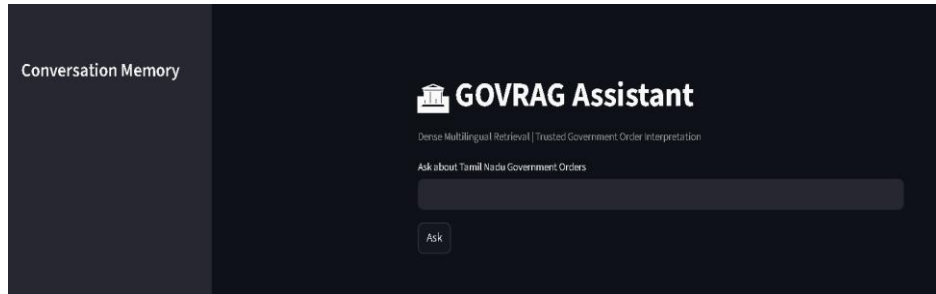


Fig. 10. GOVRAG-TN User Interface



GOVRAG Assistant

Dense Multilingual Retrieval | Trusted Government Order Interpretation

Ask about Tamil Nadu Government Orders

agriculture subsidies in 2021

Ask

Answer

GO Number: ■ **G.O.(Ms).No.230** Clause: 26

Text: Special Incentive to Sugarcane farmers

From 2016-17 to 2019-20 crushing seasons, Rs.2750/- per tonne was paid as Sugarcane price. As the same Sugarcane price was paid without any hike for the past four seasons, the Sugarcane cultivation area is declining considerably. Considering this, the Government has decided to sanction Rs.150/- per tonne of Sugarcane as "Special Incentive" directly to the Bank account of the Sugarcane farmers. By this, the sugarcane farmers will get Rs.2,900 per tonne as cane price. For this purpose, the State Government has allocated Rs.138.83 Crore during the year 2021-22 for the benefit of lakh Sugarcane farmers.

I don't have information about other specific agriculture subsidies in 2021 from the provided context.

Fig. 11. Generated citation-grounded response



Retrieved Evidence

✓ **GO: G.O. (Ms) No.10**
✗ **Clause: 3**

From the Director, Department of Agricultural Marketing and Agri Business, Chennai-32 Letter No. TNSFAC/874/2021 dated 30.11.2021. ORDER:- In Agriculture Budget 2021-2022, the Hon'ble Minister for Agriculture and Farmers Welfare has made the following announcement:-

"Action will be taken to encourage value addition in minor millets and ensure remunerative price to farmers. Minor Millet rice will b...

✓ **GO: G.O. (Ms) No.10**
✗ **Clause: 3**

From the Director, Department of Agricultural Marketing and Agri Business, Chennai-32 Letter No. TNSFAC/874/2021 dated 30.11.2021. ORDER:- In Agriculture Budget 2021-2022, the Hon'ble Minister for Agriculture and Farmers Welfare has made the following announcement:-

"Action will be taken to encourage value addition in minor millets and ensure remunerative price to farmers. Minor Millet rice will b...

Fig. 12. Retrieved Supporting Evidence

CHAPTER 4

RESULTS

The performance of the GOVRAG-TN framework is evaluated across multiple retrieval configurations and analyzed using both standard information retrieval metrics and domain-specific measures. The goal of this evaluation is not merely to report numerical improvements, but to understand how different architectural choices—particularly hybrid retrieval, reranking, and multi-document reasoning—affect the overall system behavior.

4.1 Retrieval Comparison

The retrieval performance across configurations is summarized in Table 1.

Table 1 Retrieval Performance Across Methods

Method	Recall@5	Recall@10	Recall@20	MAP	nDCG@10	MRR	GO Recall
Dense	0.52	0.60	0.64	0.402	0.489	0.471	0.64
Dense + Rerank	0.44	0.50	0.64	0.260	0.375	0.365	0.64
BM25	0.40	0.44	0.56	0.297	0.353	0.362	0.56
Hybrid	0.52	0.54	0.60	0.359	0.434	0.425	0.60
Hybrid +	0.46	0.50	0.60	0.277	0.390	0.369	0.60
Rerank							

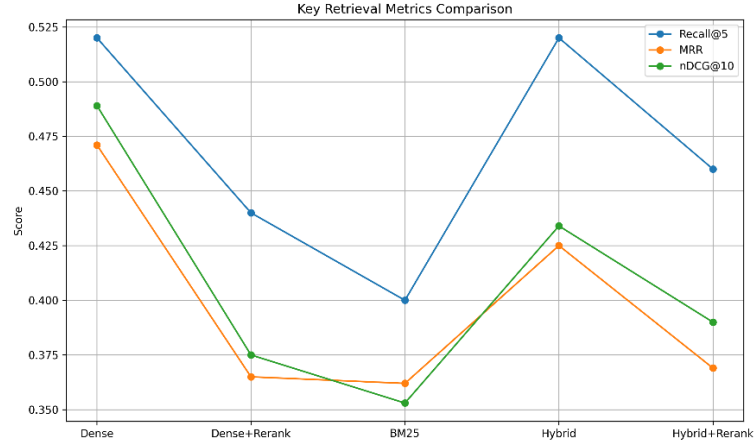


Fig. 13. Key retrieval metrics

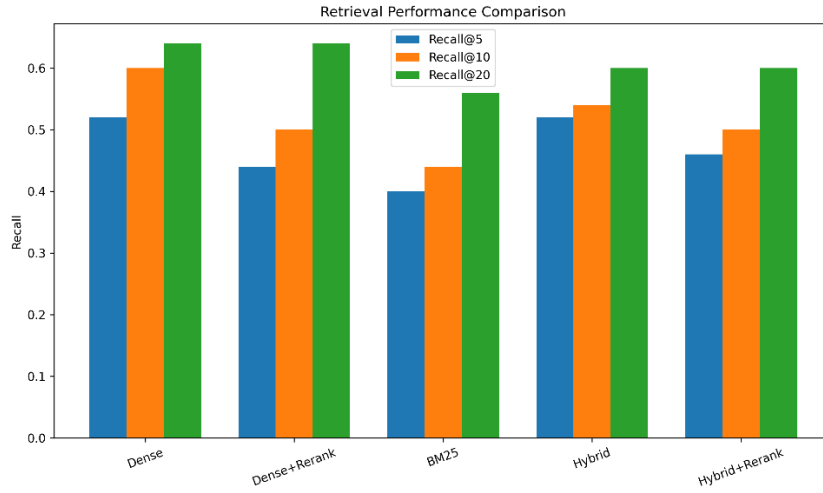


Fig. 14. Retrieval performance comparison

Fig. 13 shows that Dense semantic retrieval achieves the highest Recall@5 (0.52) and Mean Reciprocal Rank (0.471), demonstrating strong alignment between queries and relevant clauses. This highlights the effectiveness of multilingual embeddings in capturing policy semantics. BM25-based retrieval shows lower recall, particularly for queries with limited keyword overlap, indicating the limitations of lexical approaches in governance datasets.

As shown in Fig. 14, Hybrid retrieval provides balanced performance but does not consistently outperform dense retrieval, suggesting that clause-level semantic representations already capture much of the required administrative context. Reranking improves precision but leads to a slight reduction in recall, indicating that cross-encoder models may prioritize lexically similar but contextually less relevant clauses.

4.2 Clause-Level Retrieval Analysis

A notable gap exists between Government Order-level retrieval and clause-level retrieval, as shown in Fig. 15. While GO Recall reaches 0.64, clause-level recall remains significantly lower at approximately 0.10.

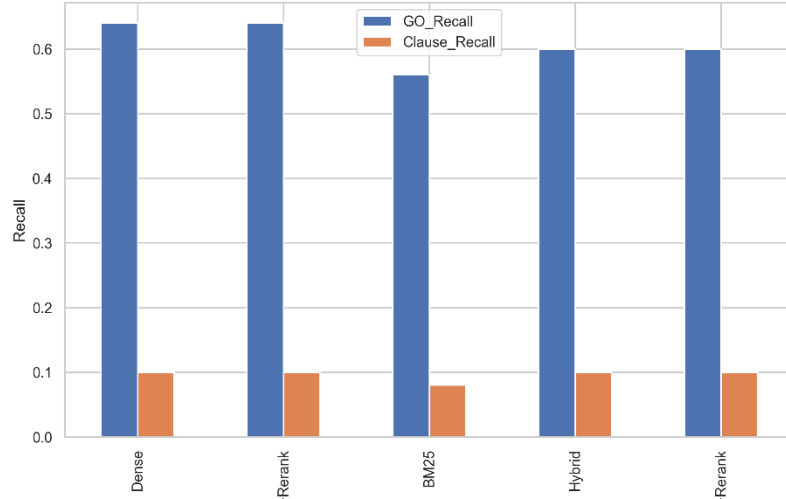


Fig. 15. GO Recall vs Clause Recall comparison

This reflects the structural complexity of Government Orders, where relevant information is distributed across clauses, annexures, and amendment sections. OCR noise and inconsistencies in clause numbering further increase the difficulty of fine-grained retrieval.

4.3 Generation Performance Evaluation

Generation performance is presented in Table 2.

Table 2 Generation Performance Across Methods

Method	Answer Accuracy	F1 Score	Similarity	GO Mention	Clause Citation	Hallucination
Dense	0.24	0.339	62.13	0.82	0.12	0.02
Dense + Rerank	0.38	0.381	67.49	0.86	0.16	0.00
BM25	0.34	0.315	63.56	0.78	0.10	0.02
Hybrid	0.26	0.343	63.15	0.86	0.16	0.02
Hybrid + Rerank	0.38	0.353	65.62	0.84	0.14	0.02

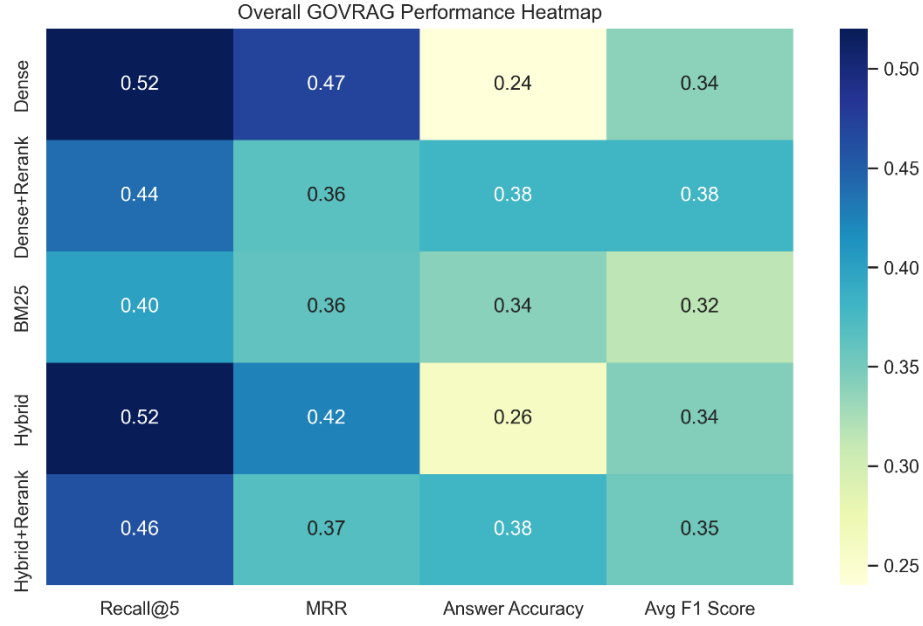


Fig. 16. Performance heatmap

Although dense retrieval achieves strong recall, as illustrated in Fig. 16, it produces lower answer accuracy (0.24). In contrast, Dense + Rerank achieves the highest answer accuracy (0.38) and improved semantic similarity, while eliminating hallucinations. This demonstrates that retrieval quality alone is insufficient; the coherence and relevance of retrieved context play a crucial role in generation.

GO mention accuracy remains consistently high (above 0.80), indicating strong grounding behavior. However, clause citation accuracy remains relatively low, highlighting challenges in precise alignment between generated responses and specific clauses.

4.4 Hallucination Analysis

Hallucination remains a critical concern in governance-oriented AI systems. As illustrated in Fig. 17, GOVRAG demonstrates very low hallucination rates (0–2%), significantly lower than typical open-domain systems.

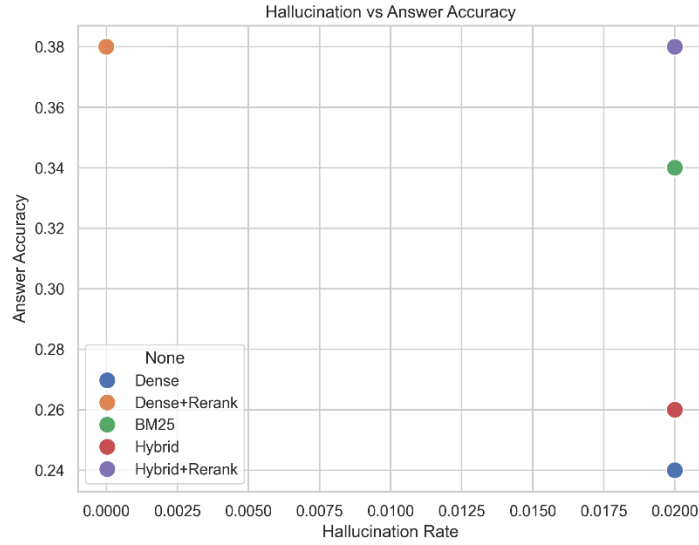


Fig. 17. Hallucination rate comparison

This reduction can be attributed to clause-aware chunking, citation-constrained prompting, multi-document evidence aggregation, and strict reliance on retrieved content. The Dense + Rerank configuration eliminates hallucinations entirely.

4.5 Impact of Multi-GO Reasoning

Many governance queries require integrating information across multiple Government Orders. Single-document retrieval often produces incomplete responses, whereas multi-document reasoning improves completeness and consistency. This capability supports amendment tracking, policy consolidation, and cross-department reasoning, particularly for procedural queries involving distributed policy information.

4.6 Temporal Policy Reasoning Evaluation

Temporal prioritization significantly improves performance for amendment-focused queries. Without temporal ordering, older Government Orders may dominate retrieval results, leading to outdated interpretations. Incorporating publication year into metadata ensures that responses reflect the most recent policy updates and improves overall accuracy.

4.7 Discussion

The experimental results highlight that interpreting governance documents differs fundamentally from traditional question-answering tasks, as effective performance requires a combination of semantic retrieval, structural awareness, and temporal

reasoning. While GOVRAG achieves a strong balance between retrieval effectiveness, generation accuracy, and hallucination control, several limitations and failure patterns are observed.

A key observation is the mismatch between retrieval performance and downstream answer quality. Dense retrieval achieves the highest early-stage recall, with Recall@5 reaching 0.52 and extending to 0.64 at Recall@20, indicating that semantic embeddings effectively capture the contextual intent of Tamil–English policy queries. However, this advantage does not directly translate to generation performance, where dense retrieval yields relatively modest Answer Accuracy (0.24) and F1 score (0.339). This reveals that high recall alone is insufficient, as the retrieved context may lack coherence or precise relevance for answer generation.

The introduction of reranking significantly improves answer reliability. The Dense + Rerank configuration increases Answer Accuracy to 0.38 and improves semantic similarity, while eliminating hallucinations entirely. This indicates that reranking plays a critical role in refining candidate passages and improving contextual alignment, even when initial retrieval metrics such as Recall@5 decrease. A similar pattern is observed in the Hybrid + Rerank configuration, where answer accuracy reaches comparable levels despite moderate retrieval performance, reinforcing the importance of ranking quality over retrieval coverage.

The hybrid retrieval approach demonstrates stable performance across different metrics, with Recall@5 at 0.52 and nDCG@10 at 0.434, suggesting that combining lexical and semantic signals improves robustness across query types. However, the corresponding improvements in answer-level metrics remain limited, indicating that retrieval diversity alone does not guarantee improved generation unless supported by effective ranking and grounding mechanisms.

Grounded generation contributes significantly to system reliability. GO mention accuracy remains consistently high across all configurations, indicating that the system effectively references source documents. Clause citation accuracy, while comparatively lower, improves with reranking, suggesting better alignment between retrieved evidence and generated outputs. The consistently low hallucination rates observed across configurations demonstrate the effectiveness of constraint-based prompting and evidence-grounded generation strategies.

Despite these strengths, several limitations persist. Clause-level retrieval remains challenging due to the distributed nature of information across multiple clauses and annexures. Additionally, OCR-induced noise and inconsistencies in document formatting can degrade retrieval quality. The clause-based chunking strategy, while effective for localized retrieval, may fail to capture inter-clause dependencies that are critical in administrative documents. Furthermore, the evaluation is conducted on a curated dataset, and system performance may vary under real-world conditions with more diverse and noisy inputs.

Overall, these observations highlight that domain-adaptive RAG systems such as GOVRAG are well-suited for governance applications, but require further enhancements in fine-grained retrieval, multi-hop reasoning, and document understanding to fully address the complexity of administrative knowledge.

CHAPTER 5

CONCLUSION

This paper presented GOVRAG-TN, a domain-adaptive and cross-lingual retrieval-augmented generation framework designed for interpreting Tamil Nadu Government Orders. By integrating clause-aware chunking, multilingual embeddings, hybrid retrieval, reranking, and evidence-grounded generation, the proposed system addresses the structural, linguistic, and temporal complexities inherent in administrative documents.

The experimental results demonstrate that retrieval effectiveness alone does not guarantee answer quality. While dense retrieval achieves strong recall performance, with Recall@5 reaching 0.52 and Recall@20 reaching 0.64, its corresponding answer accuracy remains limited at 0.24, highlighting the gap between retrieval and generation. The introduction of reranking significantly improves this alignment, with Dense + Rerank and Hybrid + Rerank configurations achieving the highest answer accuracy of 0.38 and improved semantic similarity scores exceeding 65. These findings emphasize that ranking quality and contextual coherence are critical factors in bridging retrieval outputs with reliable generation.

The system further demonstrates strong grounding capabilities. High Government Order mention rates, reaching up to 0.86, and consistent clause citation performance indicate that generated responses remain closely tied to source documents. Additionally, hallucination rates remain minimal, with some configurations achieving zero hallucinated outputs, confirming the effectiveness of constraint-driven and evidence-based generation strategies in maintaining factual reliability.

A key contribution of GOVRAG-TN lies in transforming static Government Order repositories into an interactive and queryable knowledge system. By enabling natural language querying and providing evidence-backed responses, the framework reduces reliance on manual document search and enhances accessibility for administrators, policymakers, and the general public. The ability to operate entirely offline further strengthens its applicability in governance environments where data privacy and infrastructure constraints are critical considerations.

Despite these strengths, the results also indicate areas for improvement. The gap between document-level and clause-level retrieval highlights the need for more advanced fine-grained retrieval and reasoning mechanisms. Additionally, improvements in document processing, multilingual alignment, and large-scale evaluation will be necessary to ensure robustness under real-world conditions.

Overall, GOVRAG-TN establishes a practical foundation for trustworthy AI-driven governance systems and demonstrates the effectiveness of retrieval-augmented generation in enhancing transparency, accessibility, and interpretability in public policy domains. The proposed framework provides a strong basis for future research on domain-adaptive RAG systems and their deployment in large-scale administrative settings.

CHAPTER 6

FUTURE SCOPE

Despite several strengths discussed in previous chapters, several limitations persist. First, OCR quality directly affects retrieval effectiveness, particularly in scanned documents with complex formatting. Errors introduced during text extraction can propagate through the pipeline, reducing both recall and answer fidelity. Second, the current clause-based chunking approach, while useful for localized retrieval, may fail to capture inter-clause dependencies, which are often critical in legal and administrative documents. Third, the evaluation is conducted on a curated dataset, and the observed performance—such as peak Recall@20 of 0.64 and maximum Answer Accuracy of 0.38—may vary under real-world conditions with noisier inputs.

Future work will focus on strengthening both the retrieval and reasoning capabilities of the system. Improving OCR through advanced systems such as PaddleOCR or vision-language models can enhance document fidelity at the ingestion stage. Additionally, hierarchical chunking strategies that integrate clause-level and document-level segmentation may better preserve structural relationships within Government Orders.

On the retrieval side, enhancing multilingual embeddings can improve cross-lingual alignment, particularly for Tamil–English queries. Incorporating multi-hop retrieval and reasoning mechanisms would enable the system to connect information across multiple Government Orders, supporting tasks such as amendment tracking and policy comparison. Finally, expanding the evaluation dataset and conducting user studies with administrators, policy analysts, and citizens would provide a more comprehensive assessment of system performance and usability.

With these improvements, GOVRAG-TN can evolve into a robust and scalable framework for government document understanding and public policy accessibility.

CHAPTER 7

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] V. Karpukhin, B. Oguz, S. Min, et al., “Dense Passage Retrieval for Open-Domain Question Answering,” *Proceedings of EMNLP*, pp. 6769–6781, 2020.
- [3] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” *Proceedings of ACL*, pp. 874–880, 2021.
- [4] N. Thakur, N. Reimers, J. Daxenberger, et al., “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models,” *Advances in Neural Information Processing Systems*, 2021.
- [5] L. Gao, Z. Dai, and J. Callan, “Rethinking Training Data for Dense Passage Retrieval,” *Proceedings of SIGIR*, 2021.
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of EMNLP*, pp. 3982–3992, 2019.
- [7] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, et al., “LEGAL-BERT: The Muppets Straight Out of Law School,” *Findings of EMNLP*, 2020.
- [8] X. Zheng, W. Dai, Y. Zhang, et al., “Does Pretrained Language Model Benefit Legal Judgment Prediction?” *Proceedings of ACL*, 2021.
- [9] M. Bommarito and D. Katz, “A Mathematical Approach to the Study of the United States Code,” *Artificial Intelligence and Law*, vol. 18, pp. 325–351, 2010.
- [10] A. Conneau, K. Khandelwal, N. Goyal, et al., “Unsupervised Cross-lingual Representation Learning at Scale,” *Proceedings of ACL*, 2020.
- [11] L. Wang, N. Yang, F. Wei, et al., “Text Embeddings by Weakly-Supervised Contrastive Pre-training,” *arXiv preprint arXiv:2212.03533*, 2022.

- [12] G. Izacard, E. Grave, et al., “Improving Dense Retrieval with Multi-Query Training,” *Transactions of the ACL*, 2021.
- [13] Z. Sun, X. Yu, Y. Zhang, et al., “Document Structure Aware Information Retrieval,” *Proceedings of SIGIR*, 2022.
- [14] Y. Liu, M. Ott, N. Goyal, et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [15] J. Baek, G. Kim, J. Lee, et al., “What Is Wrong With Scene Text Recognition Model Comparisons?” *Proceedings of ICCV*, 2019.
- [16] Y. Du, Z. Li, et al., “PaddleOCR: An Open-Source OCR System,” *Proceedings of ICPR*, 2021.
- [17] Y. Xu, M. Li, L. Cui, et al., “LayoutLM: Pre-training of Text and Layout for Document Image Understanding,” *Proceedings of KDD*, 2020.
- [18] G. Kim, J. Hong, et al., “Donut: Document Understanding Transformer Without OCR,” *Proceedings of ECCV*, 2022.
- [19] Y. Xu, Y. Lv, L. Cui, et al., “LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking,” *Proceedings of ACM Multimedia*, 2022.
- [20] Z. Huang, X. Wang, et al., “Multimodal Transformers for Document Understanding,” *Proceedings of CVPR*, 2021.
- [21] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” *arXiv preprint arXiv:1901.04085*, 2019.
- [22] N. Thakur, N. Reimers, et al., “Efficient Transformer-based Reranking Models for Large-scale Retrieval,” *Proceedings of SIGIR*, 2021.
- [23] Z. Ji, N. Lee, R. Frieske, et al., “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, 2023.
- [24] K. Shuster, S. Poff, et al., “Retrieval Augmentation Reduces Hallucination in Dialogue Models,” *Findings of EMNLP*, 2021.
- [25] H. Zhong, C. Guo, Z. Tu, et al., “Iteratively Questioning and Answering for Interpretable Legal Judgment Prediction,” *Proceedings of AAAI*, 2020.

- [26] T. Brown, B. Mann, N. Ryder, et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems*, 2020.
- [27] H. Touvron, L. Martin, K. Stone, et al., “LLaMA: Open and Efficient Foundation Language Models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [28] A. Jiang, A. Sablayrolles, et al., “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023.
- [29] A. Bai, X. Li, et al., “Qwen Technical Report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [30] J. Devlin, M. Chang, K. Lee, et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of NAACL*, 2019.
- [31] Y. Liu, M. Ott, et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” 2019.
- [32] J. Lin, R. Nogueira, A. Yates, “Pretrained Transformers for Text Ranking,” *arXiv preprint*, 2020.
- [33] R. Nogueira, W. Yang, et al., “Document Expansion by Query Prediction,” *Proceedings of ACL*, 2019.
- [34] C. Raffel, N. Shazeer, et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” *Journal of Machine Learning Research*, 2020.
- [35] H. Tay, D. Dehghani, et al., “Efficient Transformers: A Survey,” *ACM Computing Surveys*, 2022.
- [36] A. Radford, J. Wu, et al., “Language Models are Unsupervised Multitask Learners,” OpenAI, 2019.
- [37] J. Guu, K. Lee, Z. Tung, et al., “REALM: Retrieval-Augmented Language Model Pretraining,” *Proceedings of ICML*, 2020.
- [38] S. Min, V. Karpukhin, et al., “Dense Retrieval for Open-Domain Question Answering: A Survey,” *Transactions of the ACL*, 2021.
- [39] D. Metzler, O. Tay, et al., “Rethinking Search: Making Domain Experts out of Language Models,” *Proceedings of SIGIR*, 2021.

- [40] H. Zamani, M. Bendersky, et al., “From Neural Re-ranking to Neural Ranking,” *Proceedings of SIGIR*, 2018.
- [41] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction,” *Proceedings of SIGIR*, 2020.
- [42] M. Khattab, C. Potts, et al., “Open-Domain Question Answering with Dense Retrieval,” *Proceedings of ACL*, 2021.
- [43] Z. Xiong, et al., “Approximate Nearest Neighbor Search for Large-Scale Semantic Retrieval,” *IEEE Transactions on Knowledge and Data Engineering*, 2021.
- [44] J. Johnson, M. Douze, H. Jégou, “Billion-scale Similarity Search with FAISS,” *IEEE Transactions on Big Data*, 2019.
- [45] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, 2009.
- [46] M. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
- [47] C. Guo, H. Fan, et al., “Learning to Rank with Neural Networks,” *Proceedings of ICML*, 2019.
- [48] J. Lin and X. Ma, “A Few Brief Notes on Deep Impact, COIL, and a Conceptual Framework for Information Retrieval Techniques,” *arXiv preprint*, 2021.
- [49] T. Mikolov, I. Sutskever, et al., “Distributed Representations of Words and Phrases,” *Advances in Neural Information Processing Systems*, 2013.
- [50] T. Wolf, L. Debut, et al., “Transformers: State-of-the-Art Natural Language Processing,” *Proceedings of EMNLP System Demonstrations*, 2020.

APPENDIX 1

This section provides details on the programming language, software tools, frameworks, and libraries used in the development and implementation of the GOVRAG-TN system.

The entire system is developed using **Python** as it offers a rich ecosystem of libraries, enabling rapid development of scalable and efficient AI systems. Its readability and flexibility make it particularly suitable for complex workflows such as document processing, retrieval systems, and language model integration.

The following libraries and tools were used in this project:

Core Data Processing and Computation Libraries

- **NumPy:**
NumPy is used for handling embedding vectors, similarity computations, and numerical transformations.
- **Pandas:**
Pandas is used to organize metadata (e.g., GO number, department, clause mappings) and evaluation results.

Natural Language Processing and Embedding Libraries

- **Sentence Transformers / Embedding Models (multilingual-e5-large):**
Used for generating dense semantic embeddings of document chunks and queries. This model allows semantic similarity matching beyond keyword overlap.
- **HuggingFace Transformers:**
Provides access to pre-trained models and tokenizers. It is used for embedding generation, reranking models, and integration of transformer-based architectures within the pipeline.

Information Retrieval and Indexing

- **FAISS (Facebook AI Similarity Search):**

FAISS is used for large-scale document retrieval in offline environments.

- **BM25 (via libraries such as rank_bm25 or Elasticsearch-style implementations):**

BM25 is used for lexical retrieval, enabling exact keyword matching. It complements dense retrieval by improving precision in cases where domain-specific terms are critical.

Reranking and Retrieval Optimization

- **Cross-Encoder Models (MiniLM-L-6-v2):**

Cross-encoder models are used for reranking retrieved documents by jointly encoding query-document pairs. This improves the relevance of top-ranked results by capturing fine-grained semantic relationships.

Document Processing and OCR

- **Tesseract OCR:**

OCR tools are used to extract text from scanned Government Order PDFs. These tools support multilingual text recognition and are critical for converting image-based documents into machine-readable format.

- **PyMuPDF:**

Used for reading, parsing, and extracting content from PDF documents. These libraries help in handling document structure and converting PDFs into processable formats.

Machine Learning and Model Deployment

- **Ollama:**

Ollama is used to deploy and run the language model locally. It enables offline inference, which is essential for maintaining data privacy in governance applications.

- **Qwen2.5 (7B Model):**

This is the primary language model used for response generation. It processes structured prompts and retrieved context to generate grounded answers.

Evaluation and Analysis Tools

- **Scikit-learn:**

Used for evaluation-related computations and metrics where required. It provides utilities for performance analysis and statistical evaluation.

- **Matplotlib:**

Used for visualizing results such as retrieval performance comparisons and metric distributions. It supports generation of publication-quality plots.

Utility and Supporting Libraries

- **Regex (re module):**

Used for pattern matching and extraction of metadata such as GO numbers, dates, and clause identifiers from text.

- **JSON / Pickle:**

Used for storing intermediate data such as embeddings, indexed structures, and processed documents.

- **OS and File Handling Libraries:**

Used for managing file systems, directory traversal, and dataset organization.

APPENDIX 2

Key Implementation Modules and Representative Code Snippets

This appendix presents selected implementation snippets from the GOVRAG-TN framework. Instead of including complete source files, only representative components of each processing stage are provided to illustrate the system workflow while avoiding unnecessary implementation redundancy. The appendix focuses on the core logic behind OCR processing, clause-aware chunking, semantic retrieval, grounded generation, and evaluation.

A2.1 Configuration and System Initialization

The configuration layer defines dataset paths, OCR settings, retrieval parameters, and model initialization variables used throughout the GOVRAG-TN pipeline.

```
PROJECT_ROOT = Path("Project_Directory")
MASTER_GO_FOLDER = PROJECT_ROOT / "Dataset"
OCR_OUTPUT_DIR = PROJECT_ROOT / "rag_data/ocr"
CHUNK_DIR = PROJECT_ROOT / "rag_data/chunks"
OLLAMA_MODEL = "qwen2.5:7b"
TOP_K = 40
TESSERACT_LANG = "eng+tam"
DPI = 300
```

A2.2 OCR and Document Processing

The OCR module converts scanned Government Order PDFs into machine-readable bilingual text representations. Page-wise extraction is performed before downstream normalization and structuring.

```
def ocr_pdf(pdf_path):
    images = convert_from_path(pdf_path, dpi=250)
    extracted_pages = []
    for page in images:
        text = pytesseract.image_to_string(
            page,
            lang="eng+tam"
        )
        extracted_pages.append(text)
    return extracted_pages
```

A2.3 Text Cleaning and Normalization

After OCR extraction, the text is normalized to reduce inconsistencies while preserving clause structure and bilingual content.

```
def clean_text(text):
    text = re.sub(r'\r\n', '\n', text)
    text = re.sub(r'[ \t]{2,}', ' ', text)
    text = text.replace('fi', 'fi')
    text = text.replace('fl', 'fl')
    return text.strip()
```

A2.4 Clause-Aware Chunking

Government Orders were segmented into clause-level chunks instead of fixed-length windows. This helped preserve administrative structure during retrieval.

```
def extract_clause_chunks(text):
    parts = re.split(r"\n\s*(\d+\.\.)\s+", text)
    chunks = []
    for i in range(1, len(parts), 2):
        clause_id = parts[i]
        clause_text = parts[i + 1]
        chunks.append({
            "section_id": clause_id,
            "text": clause_text
        })
    return chunks
```

A2.5 Embedding Generation

Document chunks were converted into dense multilingual embeddings using the multilingual-e5-large model.

```
model = SentenceTransformer(
    "intfloat/multilingual-e5-large"
)
def embed_texts(texts):
    return model.encode(
        texts,
        normalize_embeddings=True
    )
```

A2.6 FAISS Vector Index Construction

The generated embeddings were indexed using FAISS to support efficient semantic retrieval across the Government Order corpus.

```

index = faiss.IndexFlatIP(dimension)
faiss.normalize_L2(embeddings)
index.add(embeddings)
faiss.write_index(index, "faiss.index")

```

A2.7 Hybrid Retrieval Mechanism

The retrieval stage combines semantic retrieval and lexical matching to improve robustness across different query styles.

```

def dense_search(query):
    query_embedding = model.encode(
        f"query: {query}",
        normalize_embeddings=True
    )
    scores, indices = index.search(
        np.array([query_embedding]),
        top_k
    )
    return indices

```

A2.8 Prompt Construction and Grounded Generation

Retrieved clauses were structured into evidence-aware prompts before response generation.

```

prompt = f"""
You are GOVRAG –
Tamil Nadu Government Order Assistant.
Use ONLY the provided evidence.
QUESTION:
{question}
CONTEXT:
{context}
FINAL ANSWER:
"""

```

A2.9 Citation-Aware Response Generation

The language model generated grounded responses while preserving Government Order references and clause information.

```

response = ollama.chat(
    model="qwen2.5:7b",
    messages=[
        {
            "role": "user",

```

```

        "content": prompt
    }
]
)

```

A2.10 Streamlit-Based User Interface

A lightweight Streamlit interface was developed for interactive querying and evidence visualization.

```

st.title(
    "GOVRAG – Tamil Nadu Government Orders Assistant"
)
query = st.text_input(
    "Ask about Government Orders"
)

```

A2.11 Retrieval Evaluation Workflow

Retrieval performance was evaluated using multiple retrieval strategies including Dense Retrieval, BM25, Hybrid Retrieval, and Hybrid + Reranking.

```

def evaluate_method(method_name):
    dense_ids = dense_retrieve(question)
    bm25_ids = bm25_retrieve(question)
    hybrid_ids = rrf_fusion(
        dense_ids,
        bm25_ids
    )
    return hybrid_ids

```

A2.12 Generation Evaluation Workflow

Generated responses were evaluated using answer correctness, citation accuracy, hallucination analysis, and GO-level correctness.

```

similarity = fuzz.partial_ratio(
    expected_answer,
    generated_answer
)
if similarity >= 70:
    answer_correct += 1

```

PAPER PUBLICATION STATUS

PLAGIARISM REPORT